Guidance of Autoregressive Protein Language Models

Mikkel Niklas Rasmussen[1, 2] and Christian Hvilshøj[1]

[1]Department of Applied Mathematics and Computer Science, Cognitive Systems
[2]Department of Health Technology, Bioinformatics

Abstract

In this project, you will investigate guided generation of sequential data using autoregressive models. You will implement and train generative models on a protein dataset containing sequences from three protein families. You will then extend these models to predict the family of a given protein sequence. Finally, you will combine the generative and predictive models through Bayes' rule to steer sequence generation towards a chosen family.

Key concepts: Generative modeling, sequential data, autoregressive modeling, multilayer perceptrons (MLPs), recurrent neural networks (RNNs), causal Transformers, classifier and classifier-free guidance.

1 Introduction

Modern protein language models can generate realistic protein sequences by learning to approximate the distribution of naturally occurring proteins. However, in protein design we are often interested in generating sequences with particular properties, such as belonging to a specific protein family, having a desired structural characteristic, or satisfying a distribution of experimentally measured properties. This project will investigate how guidance can be used to steer autoregressive generative models towards desired protein sequences. The project will progressively develop this idea, from simple multi-layer perceptron (MLP) models to recurrent neural networks (RNNs) or Transformer-based models.

You will be working with *sequential* data of the form:

$$\mathbf{x} = (x_1, \ldots, x_L) = \mathbf{x}_{1:L}, \tag{1}$$

where $\mathbf{x}_{1:L} \in \mathbb{A}^L$ represents a protein sequence of length L over the token alphabet $\mathbb{A}$, which in this project is the 20 canonical amino acids. We then define autoregressive models (ARMs) over the discrete state space $\mathbb{A}^L$ using:

$$p(\mathbf{x}_{1:L}) = \prod_{i=1}^{L} p(x_i \mid \mathbf{x}_{<i}), \tag{2}$$

with the shorthand notation $\mathbf{x}_{<i} = (x_1, \ldots, x_{i-1})$ and $p(x_1|x_{<1}) = p(x_1)$. Furthermore, let $y \in \mathcal{Y}$ denote some property of interest, which could be catalytic activity, mutation effects, the protein family, etc. and assume the following joint distribution over both $\mathbf{x}$ and y:

$$p(\mathbf{x}, y) = p(\mathbf{x})p(y|\mathbf{x}), \tag{3}$$

where we will refer to $p(y|\mathbf{x})$ as the *predictive* model. Using Bayes' rule, we can combine the generative ARM and the predictive model in order to obtain a distribution over sequences conditioned on a desired property:

$$p(x_i \mid \mathbf{x}_{<i}, y) = \frac{p(y \mid x_i, \mathbf{x}_{<i})p(x_i \mid \mathbf{x}_{<i})}{p(y \mid \mathbf{x}_{<i})} \tag{4}$$

$$= \frac{p(y \mid \mathbf{x}_{\leq i})p(x_i \mid \mathbf{x}_{<i})}{p(y \mid \mathbf{x}_{<i})} \tag{5}$$

$$\propto \underbrace{p(y \mid \mathbf{x}_{\leq i})}_{\text{guidance}} \underbrace{p(x_i \mid \mathbf{x}_{<i})}_{\text{autoregressive}}. \tag{6}$$

In this project, you will investigate how the desired conditional distribution can be translated into a sequential generation procedure, allowing sequences to be generated while conditioning on a specified protein family. This stage will establish the central concept used throughout the remainder of the project:

$$\boxed{\text{generative model + predictive model} \longrightarrow \text{guided generation}}$$

1.1 Data

The project uses data covering the three different protein families, acylphosphatase (AcyP), histidine-containing phosphocarrier protein (HPr), and HU DNA-binding proteins (HU). The data has already been divided into a train, validation and test set. This has been done since protein sequences are related through evolution. Even though the origin of the protein sequences can be from different species, the sequence similarity can still be very high. In order to make sure data leakage does not occur, the protein sequences are first clustered based on sequence similarity and then divided into train, validation, and test sets, so that very similar protein sequences occur in the same split.

1.2 Project outline

Autoregressive models. For the ARMs, you will approximate the distribution $p(\mathbf{x})$ using a trained neural network with parameters $\theta \in \mathbb{R}^d$:

$$p(\mathbf{x}) \approx p_\theta(\mathbf{x}_{1:L}) = \prod_{i=1}^{L} p_\theta(x_i \mid \mathbf{x}_{<i}), \tag{7}$$

where the network will take the form of increasingly more and more complex architectures.

Predictive model. The predictive model will also be approximated using a neural network with parameters $\phi \in \mathbb{R}^k$. Importantly, to compute Eq.(6) the predictive model needs to be trained on partially finished sequences $\mathbf{x}_{\leq i}$, such that it can predict the probability of $y \in \{1, 2, 3\}$ at each step of the generation:

$$p_\phi(y \mid \mathbf{x}_{\leq i}) \quad \forall i \in \{1, \ldots, L\}, \tag{8}$$

This is also referred to as a *noisy* predictor, as it can predict the properties of y given noisy/incomplete sequences. We suggest reusing the learned generator $p_\theta(\mathbf{x})$ for the encoding of the sequences and thus only train a prediction "head" such that $\theta \subset \phi$.

Classifier guidance. In order to do guidance as described in Eq. (6), we recommend computing it in log-space:

$$\log \tilde{p}(x_i \mid \mathbf{x}_{<i}, y) \approx \log p_\theta(x_i \mid \mathbf{x}_{<i}) + \lambda \log p_\phi(y \mid \mathbf{x}_{\leq i}), \tag{9}$$

with a tunable hyperparameter λ controlling the guidance strength and $\tilde{p}$ depicting the *unnormalized* token probabilities. By combining the generative and predictive model, the predictive term provides information on how likely the partially generated sequence will eventually correspond to the desired property. **Note:** for the guidance of token x_i, we need to compute $p(y \mid \mathbf{x}_{\leq i}) = p(y \mid \mathbf{x}_{<i}, x_i)$ thus we need to evaluate the predictor of each possible value of $x_i \in \mathbb{A}$.

Classifier-free guidance (CFG). Eq. (9) requires the predictor $p_\phi(y \mid \mathbf{x}_{\leq i})$ for *every*

candidate token $x_i \in \mathbb{A}$ at every generation step, i.e. $|\mathbb{A}|$ evaluations of the predictor per token. *Classifier-free guidance* (CFG) [1] avoids this by training a single network that models both the conditional, $p_\theta(x_i \mid \mathbf{x}_{<i}, y)$, and the unconditional, $p_\theta(x_i \mid \mathbf{x}_{<i})$, next-token distribution. During training, the label y is replaced by a "null" label $\varnothing$ with probability p_{drop} (e.g. 0.1–0.2), so the network learns both distributions. The label can be provided to the network, e.g., as an additional (learned) embedding or as a first token. The idea is that Bayes' rule gives an implicit predictor,

$$p(y \mid \mathbf{x}_{\leq i}) \;\propto\; \frac{p_\theta(x_i \mid \mathbf{x}_{<i}, y)}{p_\theta(x_i \mid \mathbf{x}_{<i}, \varnothing)}, \tag{10}$$

where the proportionality is in x_i (the remaining factor $p(y \mid \mathbf{x}_{<i})$ does not depend on x_i and is absorbed by normalisation). Inserting this into Eq. (9) with $\lambda = 1 + w$ gives:

$$\log \tilde{p}(x_i \mid \mathbf{x}_{<i}, y) = \log p_\theta(x_i \mid \mathbf{x}_{<i}, \varnothing) + (1 + w) \log \frac{p_\theta(x_i \mid \mathbf{x}_{<i}, y)}{p_\theta(x_i \mid \mathbf{x}_{<i}, \varnothing)} + \text{const} \tag{11}$$

$$\propto (1 + w) \log p_\theta(x_i \mid \mathbf{x}_{<i}, y) - w \log p_\theta(x_i \mid \mathbf{x}_{<i}, \varnothing), \tag{12}$$

where $w = 0$ recovers sampling from the conditional model (corresponding to $\lambda = 1$ in Eq. (9)), and $w > 0$ extrapolates further in the direction in which the label changes the predictions. CFG avoids training a separate predictor but requires conditional and unconditional model evaluations at each generation step.

Remark. For $\lambda = 1$ (or $w = 0$ for CFG) and using exact distributions, guidance recovers the conditional distribution exactly using Bayes' rule, as described in Eq. (6). Empirically, researchers have found that using $\lambda > 1$ (or $w > 0$), i.e., a higher guidance strength, can leads to better results. Further, since Eqs. (9) and (12) define a *per-token* guidance rule that scores $\mathbf{x}_{\leq i}$ rather than the completed sequence; it does not in general sample from $p(\mathbf{x})\, p(y \mid \mathbf{x})^{\lambda}$. Its usefulness is therefore assessed empirically, through the trade-off between family membership and structural plausibility as λ (or w) varies.

1.3 Project goals

Phase 1: Autoregressive models.

- Implement and train a simple MLP language model akin to Figure 1 in [2] that takes in the three previous tokens (amino acids) and predicts the next, using a deep learning framework such as PyTorch. The model architecture and hyperparameters are up to you, but clearly state and justify your choices in the report.
- Implement and train a recurrent neural network (RNN) model **or** a causal decoder-only multi-head attention Transformer model [3]. For the RNN choose between the vanilla RNN, Gated Recurrent Unit (GRU) or Long Short-Term Memory (LSTM). Like the MLP, the specific model design is up to you but needs to be explained and justified in the report.
- Evaluate and visualize the performance of these models by folding the generated sequences with ESMFold [4] and presenting relevant metrics (code for evaluation will be provided).

Phase 2: Classifier guidance

- Select your trained RNN or Transformer, freeze the model weights, and then re-train a classification head on top of the frozen model to predict the protein family. Remember, the head needs to be a noisy predictor such that it learns to predict the family membership even when it has seen only part of a sequence.
- Evaluate the overall performance of the predictive model on clean and partial finished sequences.
- Perform guided generation towards one of the protein families using classifier guidance. Try varying λ to see the effect of guidance.

Phase 3: Classifier-free guidance

- Train a conditioned version of your selected autoregressive architecture, using label dropout so that the same network also learns both unconditional and conditional generation. Use this model to investigate classifier-free guidance.
- Evaluate and visualize the performance of the guided and unguided models (classifier and classifier-free guidance). Investigate whether guidance increases membership in the target

family while retaining the model's ability to generate realistic protein structures, and to characterize the trade-off between the two. Use the predictive model to report predicted target-family membership. Discuss the limitations of evaluating classifier-guided samples using the same predictor that guided their generation.

- Discuss what the difficulties are when evaluating generated data. How do we know the generated data is "good"?

1.4 Learning outcomes

After completing the project, you should be able to:

- Explain autoregressive sequence modeling and relate next-token prediction and its training objective to the probability of a complete sequence.
- Implement and train autoregressive neural networks, and justify architectural choices in terms of their ability to represent dependencies in sequential data.
- Explain why classifier guidance requires predictions from partial sequences, and assess how predictive performance depends on the amount of observed context.
- Explain classifier guidance using Bayes' rule and distinguish the exact probabilistic identity from its implementation using learned models.
- Explain how classifier-free guidance combines conditional and unconditional predictions, and compare its training and sampling requirements with classifier guidance.
- Analyze how guidance strength affects target-family assignment and structural plausibility, and critically assess what the evaluation metrics support.

References

1. Ho J and Salimans T. Classifier-Free Diffusion Guidance. 2022. arXiv: 2207.12598 [cs.LG]. URL: https://arxiv.org/abs/2207.12598.
2. Bengio Y, Ducharme R, Vincent P, and Jauvin C. A Neural Probabilistic Language Model. Journal of Machine Learning Research 2003;3:1137–55.
3. Vaswani A, Shazeer N, Parmar N, et al. Attention Is All You Need. 2023. arXiv: 1706.03762 [cs.CL]. URL: https://arxiv.org/abs/1706.03762.
4. Lin Z, Akin H, Rao R, et al. Evolutionary-scale prediction of atomic-level protein structure with a language model. Science 2023;379:1123–1130.